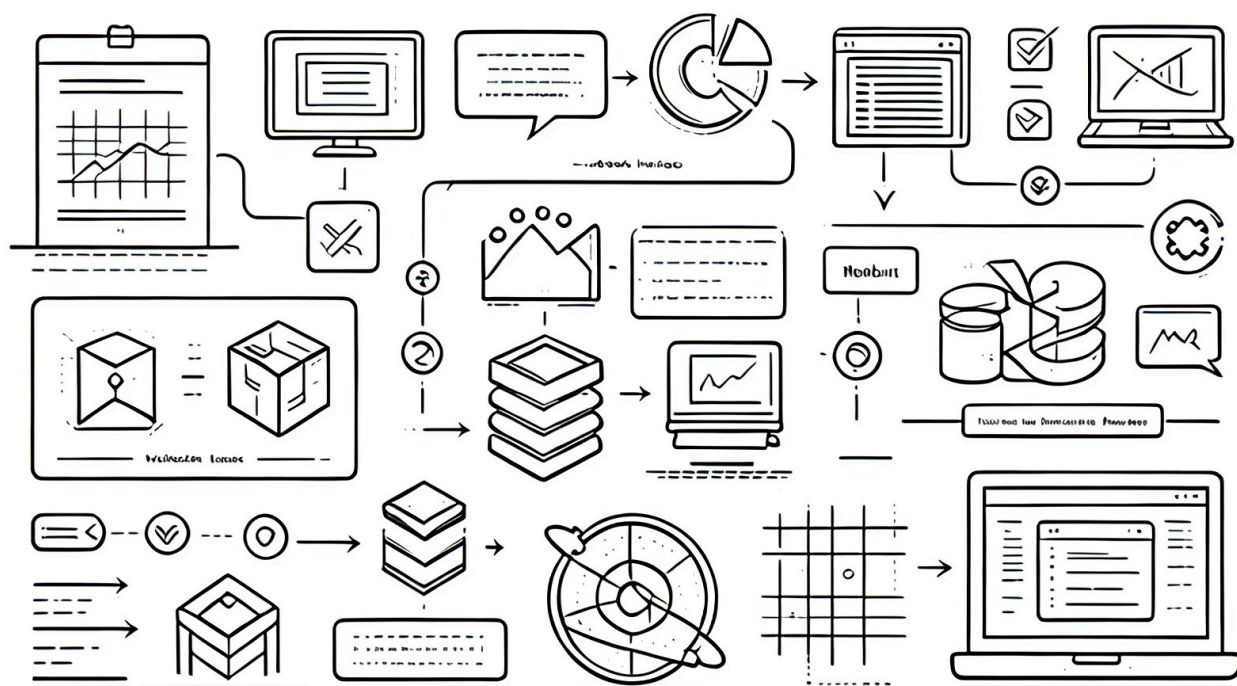




7天学会Haskell

AI时代下的函数式编程

*Haskell China*团队
编著

*Haskell China*出品

豆包AI生成

前言

关于我们

Haskell China 是专注于在中国推广 Haskell 纯函数式编程语言的本土技术社区，致力于普及函数式编程思想、沉淀实战技术经验、探索前沿学术与工程应用，面向广大编程爱好者、一线开发者、科研工作者与技术学习者，搭建开放、纯粹、互助的中文 Haskell 交流与共建平台。

社区官方网站：<https://haskell.org.cn>。平台整合了体系化的 Haskell 学习与研发资源，覆盖零基础入门教程、经典中外书籍、海内外名校公开课、GHC、Cabal、Stack 全套工具链使用指南，同时持续输出大量原创深度技术内容，涵盖代数效应、依赖类型、线性类型、惰性求值、可微分编程、分布式系统、神经符号计算、量子计算、联邦学习等前沿技术方向，兼顾新手入门学习与资深开发者、科研人员的深度深耕需求，全方位助力国内学习者掌握 Haskell 核心能力并落地工程与学术应用。

一、Haskell 在中国的整体发展概况

Haskell 传入中国已有十余年，国内生态整体呈现**小众但稳健、学术先行、工业界逐步落地**的发展特征，历经多个阶段性成长过程。

2010年前后为社区萌芽阶段。国内最早一批 Haskell 爱好者依托邮件列表、开源社区频道、线上社群自发聚集，自发开展文档翻译、技术笔记分享、线上技术研讨等活动。同时《Haskell 趣学指南》等经典中文译作问世，填补了国内中文学习资料的空白，为 Haskell 在国内的初步普及奠定了基础。

2015年至今，学术圈层持续深耕发展。国内清华、北大、复旦、上交等顶尖高校，陆续开设函数式编程、类型系统、形式化验证、程序逻辑等相关课程。众多高校实验室依托 Haskell、Agda 等函数式工具，深耕依赖类型、程序验证、编译器构造、形式化建模等前沿研究领域。国内学者在 ICFP、POPL 等国际顶级函数式编程会议的参与度持续提升，Haskell 已然成为国内理论计算机科学研究的核心工具语言之一。

2018年至今，工业应用逐步落地萌芽。相较于欧美成熟的工业落地生态，国内 Haskell 工程化应用仍处于小众阶段，但发展势头稳定。目前金融量化、区块链、云原生基础架构、高可靠服务、前端全栈开发等领域的创业团队与技术团队，已开始在生产环境中落地 Haskell，依托其高可靠、强可验证、高并发的特性，搭建交易系统、风控引擎、分布式协调服务、定制化编译器与解释器等核心基础设施。

2020年至今，中文社区走向规范化运营。随着国内函数式编程认知的持续普及，零散的个人交流逐步整合为体系化的社区生态。线上常态化技术交流社群、中文原创技术博客、公开教学分享持续增多，以 Haskell China 官方网站为核心的资源阵地正式成型。同时国内定期举办线上线下技术沙龙、Hackathon、专题研讨活动，打通学术研究与工业落地的壁垒，持续完善本土 Haskell 技术生态。

二、AI 时代下 Haskell 的发展机遇与挑战

人工智能技术的高速迭代，重塑了计算机编程语言的应用场景与价值体系，也为长期小众的 Haskell 带来了全新的发展机遇，同时也暴露了其适配主流工业生态的诸多短板。

（一）核心发展机遇

第一，适配高可信 AI 的发展需求。当下 AI 模型黑盒问题、逻辑漏洞、安全风险愈发突出，行业对**可解释、可验证、高安全**的可信 AI 需求激增。Haskell 凭借强静态类型、纯函数、不可变数据、编译期校验的核心特性，能够从代码底层规避大量逻辑错误，天然适配 AI 决策逻辑验证、关键场景 AI 安全校验、神经符号推理等场景，是高可信 AI 研发的优质工具。

第二，高度契合 AI 理论与前沿研究。Haskell 极致的数学抽象能力、高阶函数、代数数据类型、模式匹配等语法特性，能够无缝映射各类数学算法与学术理论，大幅降低理论落地为代码的成本，在概率编程、强化学习、神经符号 AI、可微分编程、量子机器学习等前沿研究领域，具备主流编程语言无法比拟的表达优势。

第三，适配大规模并行计算场景。Haskell 天然支持无副作用编程，彻底规避并发竞态问题，搭配成熟的 GHC 并行运行时，能够高效支撑分布式训练、批量数据处理、并行推理等 AI 基础设施搭建，适配大模型时代的海量算力调度需求。

第四，开发生态形成正向循环。随着大模型代码能力的迭代升级，AI 工具对 Haskell 代码的生成、补全、纠错能力持续提升，有效降低了 Haskell 的入门门槛与工程开发成本；同时 Haskell 的强类型约束，也能反向校验 AI 生成代码的规范性与正确性，形成良性互补。

（二）现存核心挑战

其一，主流 AI 生态适配不足。当前全球 AI 生态以 Python 为绝对核心，拥有完善的深度学习框架、预训练模型库、MLOps 工具链。而 Haskell 缺乏成熟的工业级 AI 配套库，与主流 AI 工具、平台的兼容性较差，大幅提升了 Haskell 落地 AI 工程场景的成本。

其二，技术人才储备稀缺。Haskell 独特的函数式编程范式，与国内主流的命令式编程思维差异极大，学习曲线陡峭。国内 AI 从业者多以 Python 技术栈为主，转型成本较高，专业 Haskell 研发与科研人才体量小，难以满足规模化工业落地的人才需求。

其三，运行性能存在不确定性。Haskell 惰性求值带来了极致的代码简洁性与抽象能力，但也会带来不可预测的内存占用与延迟问题，难以适配 AI 实时推理、超高吞吐等严苛的生产场景，限制了其工业规模化应用。

其四，主流工程 workflow 融合度低。Haskell 与 Jupyter、MLflow、Kubeflow 等主流 AI 开发、运维工具链集成难度高，跨语言数据交互、服务适配成本高，无法无缝融入现有标准化 AI 工程体系。

三、社区使命与愿景

Haskell China 始终以**降低 Haskell 学习门槛、完善中文技术生态、打通学术与工业落地壁垒、推动函数式编程在国内高质量发展**为核心使命。我们致力于让更多开发者、科研从业者认识 Haskell、掌握 Haskell、应用 Haskell，充分发挥其在形式化验证、可信计算、前沿 AI 研究、高可靠系统开发等场景的独特价值。

我们诚挚欢迎所有热爱函数式编程、深耕前沿技术、追求代码严谨性与可靠性的技术爱好者加入社区，共同交流学习、沉淀技术、共建国内 Haskell 生态。

官方网站：<https://haskell.org.cn>

联系邮箱：team@haskell.org.cn

专栏定位：一篇真正面向初学者、但又尽可能专业严谨的 Haskell 深度入门长文

阅读对象：零基础读者、具备其他语言经验但首次接触函数式编程的开发者、希望系统理解 Haskell 思维方式的工程师

目标：通过 7 天的结构化学习路径，完成从语法入门、类型理解、递归与高阶函数，到代数数据类型、类型类、惰性求值、IO、Monad 和工程化实践的完整入门。

写作原则：不仅讲“怎么写”，更讲“为什么这样设计”；不仅给出代码，更解释代码背后的抽象和工程意义。

摘要

Haskell 是一门极具代表性的纯函数式编程语言。它以强静态类型、类型推导、惰性求值、不可变数据、代数数据类型和类型类系统闻名。对于初学者来说，Haskell 的最大门槛往往不是语法本身，而是它彻底不同于命令式语言的思维方式：在这里，程序不再主要由赋值、循环和状态修改组成，而更多是由表达式、函数组合、模式匹配和类型驱动设计构成。

本文将以“7 天入门”的路线，从最基础的表达式、函数和类型开始，逐步讲解列表、元组、递归、高阶函数、柯里化、lambda、fold、代数数据类型、记录语法、Maybe、Either、类型类、Functor、Applicative、Monad、惰性求值、IO、副作用边界、模块化和 Cabal/Stack 项目结构。文章会尽量保持“正式专栏”的专业风格，同时通过大量代码示例帮助你建立直觉，避免只会背概念而不会写程序。

如果你希望的不只是“知道 Haskell 有这些东西”，而是希望真正理解：**为什么 Haskell 要这样设计、这些设计如何帮助我们写出更可靠的程序、以及它如何反过来提升我们对其他语言的理解**，那么这篇文章就是为你准备的。

关键词

Haskell、函数式编程、纯函数、惰性求值、类型系统、代数数据类型、类型类、Functor、Applicative、Monad、IO、递归、模式匹配、Cabal、Stack

目录

- 一、为什么要学 Haskell?
 - 二、环境准备：从安装到 REPL
 - 三、7 天学习路线总览
 - Day 1: 表达式、函数、类型系统基础
 - Day 2: 列表、元组、模式匹配与递归
 - Day 3: 高阶函数、柯里化、lambda、组合与 fold
 - Day 4: 代数数据类型、记录、类型别名、Maybe 与 Either
 - Day 5: 类型类、Functor、Applicative、Monad 的直觉与实践
 - Day 6: IO、do 语法、文件读写与纯/不纯边界
 - Day 7: 模块化、Cabal/Stack、项目结构与实战案例
 - 四、惰性求值深讲：Haskell 为什么能处理无限列表?
 - 五、Haskell 初学者常见误区与踩坑指南
 - 六、如何高效学习 Haskell：方法、建议与路线图
 - 七、结语：Haskell 真正改变你的是什么?
 - 附录 A: 完整代码清单
 - 附录 B: 练习题与参考答案
-

一、为什么要学 Haskell?

在正式进入语法之前，我们先回答一个根本问题：为什么值得投入时间学习 Haskell?

这不是一句鸡汤式的“因为它高级”，也不是因为“学会 Haskell 就显得很厉害”。真正的原因在于，Haskell 会迫使你重新思考程序设计的本质。

1.1 Haskell 的几个核心标签

Haskell 通常会被贴上以下几个标签：

- 纯函数式语言
- 强静态类型语言
- 类型推导能力很强
- 默认不可变
- 惰性求值
- 代数数据类型
- 类型类系统
- 高抽象能力

这些标签不是孤立存在的，而是共同构成了 Haskell 的设计哲学。

例如：

- 因为它强调纯函数，所以副作用必须被显式建模。
 - 因为它强调类型系统，所以错误处理和数据建模会尽量通过类型表达。
 - 因为它惰性求值，所以无限列表和按需计算自然成立。
 - 因为它抽象能力强，所以很多看似复杂的问题可以通过统一的组合方式解决。
-

1.2 Haskell 学到的不是“语法”，而是“程序观”

很多语言的学习，本质上是掌握一套工具和 API。

而 Haskell 的学习，更像是一次“程序观”的升级。

你会逐渐开始思考：

- 为什么可变状态会让程序难以推理？
- 为什么副作用需要被隔离？
- 为什么类型可以充当设计文档？
- 为什么显式错误处理比运行时崩溃更可靠？
- 为什么组合比继承在很多场景下更自然？

这些问题，会反过来影响你使用其他语言的方式。

1.3 学 Haskell 的现实价值

即便你未来不一定长期用 Haskell 做业务开发，它依然有现实价值：

1) 提升代码设计能力

你会更重视函数边界、数据建模和错误显式化。

2) 提升类型系统理解

这对学习 Rust、Scala、F#、OCaml、TypeScript、Elm 等语言都非常有帮助。

3) 提升抽象能力

你会更自然地写出可组合、可测试、可推理的代码。

4) 提升工程判断力

你会更敏感地发现“哪些设计会导致隐式状态和难以维护的逻辑”。

1.4 Haskell 难吗？

坦率地说：**难**，但不是因为语法**难**，而是因为思维方式**不同**。

对于大多数初学者来说，真正难的点通常是：

- 不习惯没有“变量修改”
- 不习惯没有传统循环
- 不习惯所有东西都围绕表达式和函数展开
- 不习惯把错误处理写进类型
- 不习惯 `IO String` 和 `String` 是两回事
- 不习惯 `Maybe`、`Either`、`Functor`、`Monad` 这些抽象

但一旦跨过最初的认知鸿沟，你会发现 Haskell 其实非常一致、非常优雅。

二、环境准备：从安装到 REPL

学习 Haskell，最重要的工具不是 IDE，而是 **GHCI**。

在 REPL 里不断试表达式、看类型、验证猜想，是最有效的学习方式。

2.1 安装工具链

推荐使用 **GHCup** 安装 Haskell 工具链。

它通常会帮你安装：

- **ghc**：Glasgow Haskell Compiler
- **ghci**：交互式环境
- **cabal**：构建工具
- **stack**：另一套常见构建工具
- **hls**：Haskell Language Server

安装完成后验证：

```
ghc --version
ghci --version
cabal --version
stack --version
```

2.2 进入 GHCi

```
ghci
```

你会看到类似：

```
GHCi, version 9.x.x
Prelude>
```

Prelude 是 Haskell 默认导入的基础模块，包含大量常用函数。

试几个表达式：

```
Prelude> 1 + 2
3

Prelude> "Hello, " ++ "Haskell"
```

```
"Hello, Haskell"
```

```
Prelude> not True  
False
```

查看类型：

```
Prelude> :t 1  
1 :: Num a => a  
  
Prelude> :t "abc"  
"abc" :: String  
  
Prelude> :t not  
not :: Bool -> Bool
```

退出：

```
:q
```

2.3 推荐编辑器与开发体验

推荐：

- VS Code + Haskell 扩展
- 安装 Haskell Language Server
- 使用 `cabal` 或 `stack` 管理项目

如果你是初学者，优先保证这三点：

1. 能运行 GHCi
 2. 能查看类型
 3. 能编译一个简单项目
-

三、7 天学习路线总览

这篇文章的结构是“7 天路线式”，但请注意：

7 天不是速成神话，而是帮助你建立认知框架。

你完全可以把 7 天拉长成 2 周、1 个月，甚至更久。

Day 1: 表达式、函数、类型系统基础

目标：理解 Haskell 最基本的语法和“表达式优先”的思维。

Day 2: 列表、元组、模式匹配与递归

目标：掌握最常见的数据结构与最基础的数据处理方式。

Day 3: 高阶函数、柯里化、lambda、组合与 fold

目标：正式进入函数式编程的核心世界。

Day 4: 代数数据类型、记录、Maybe 与 Either

目标：开始理解“类型驱动设计”。

Day 5: 类型类、Functor、Applicative、Monad

目标：建立对 Haskell 抽象系统的第一层直觉。

Day 6: IO、do 语法、文件读写与纯/不纯边界

目标：理解 Haskell 如何显式管理副作用。

Day 7: 模块化、Cabal/Stack、项目结构与实战案例

目标：从“会写语法”走向“会组织项目”。

Day 1: 表达式、函数、类型系统基础

1. Haskell 是表达式语言

在很多命令式语言里，程序主要由“语句”推动：

- 赋值
- 循环
- 条件分支
- 函数调用

而在 Haskell 中，首先要建立一个关键意识：

Haskell 中几乎一切都围绕表达式展开。

表达式会产生值。

例如：

```
1 + 2
```

结果是：

```
3
```

再比如：

```
if 3 > 2 then "yes" else "no"
```

结果是：

```
"yes"
```

注意：`if` 在 Haskell 中不是“执行一个分支语句”，而是“选择一个表达式的值”。
因此 `if` 必须带 `else`。

```
x = if 3 > 2 then 1 else 0
```

2. 值绑定与函数定义

2.1 值绑定

```
x = 10  
y = 20  
z = x + y
```

这里的 `x` 不是“可变变量”，而是一个名字绑定到某个值。

2.2 函数定义

```
add a b = a + b
```

调用：

```
add 3 4
```

结果：

```
7
```

Haskell 的函数调用使用空格传参，而不是逗号和括号。

```
add 3 4
```

不是：

```
add(3, 4)
```

括号只在控制优先级时使用：

```
add (1 + 2) (3 + 4)
```

3. 基础类型

用 `:t` 查看类型：

```
Prelude> :t True
True :: Bool

Prelude> :t 'a'
'a' :: Char

Prelude> :t "hello"
"hello" :: String
```

常见基础类型：

- `Int`：固定精度整数
- `Integer`：任意精度整数
- `Float`：单精度浮点数
- `Double`：双精度浮点数
- `Bool`：布尔值
- `Char`：字符
- `String`：字符串

示例：

```
a :: Int
a = 42

b :: Double
b = 3.14

c :: Bool
c = False

d :: Char
d = 'H'

e :: String
e = "Haskell"
```

4. 类型签名

虽然 Haskell 能自动推导类型，但写类型签名是非常好的习惯。

```
add :: Int -> Int -> Int
add a b = a + b
```

再看几个例子：

```
square :: Int -> Int
square x = x * x

isEven :: Int -> Bool
isEven x = x `mod` 2 == 0

greet :: String -> String
greet name = "Hello, " ++ name
```

如何理解 `Int -> Int -> Int`?

它表示：

- 接收一个 `Int`
- 返回一个函数，这个函数再接收一个 `Int`
- 最终返回一个 `Int`

这背后就是 Haskell 的柯里化思想。

5. 中缀与前缀函数

运算符本质上也是函数。

```
(+) 1 2
```

等价于：

```
1 + 2
```

普通函数也可以变成中缀形式：

```
mod 10 3
```

等价于：

```
10 `mod` 3
```

例子：

```
isMultipleOf :: Int -> Int -> Bool
isMultipleOf a b = a `mod` b == 0
```

调用：

```
12 `isMultipleOf` 3
```

6. 不可变性：Haskell 中没有“变量更新”

在命令式语言中，变量通常意味着“一个可以被不断修改的盒子”。

而在 Haskell 中：

```
x = 10
```

更像是“给值 10 起了个名字”。

你不能在同一作用域里反复修改它：

```
x = 10
x = 20
```

这是不允许的。

不可变性带来的好处包括：

- 更容易推理程序
- 减少共享状态带来的 bug
- 更适合并发
- 更接近数学函数的语义

7. 注释

单行注释：

```
-- 这是注释
```

多行注释：

```
{-  
这是多行注释  
可以写很多行  
-}
```

8. Day 1 练习

练习 1: 双倍函数

```
double :: Int -> Int  
double x = x * 2
```

练习 2: 三倍函数

```
triple :: Int -> Int  
triple x = x * 3
```

练习 3: 判断是否成年

```
isAdult :: Int -> Bool  
isAdult age = age >= 18
```

练习 4: 拼接欢迎语

```
welcome :: String -> String  
welcome name = "Welcome, " ++ name ++ "!"
```

Day 2: 列表、元组、模式匹配与递归

列表是 Haskell 中最重要的数据结构之一，而模式匹配和递归则是处理列表的基础武器。

1. 列表基础

列表写法：

```
[1, 2, 3, 4]
```

字符串其实也是字符列表：

```
"abc" == ['a', 'b', 'c']
```

列表元素必须同类型：

```
[1, 2, 3]      -- 合法  
[True, False] -- 合法
```

```
[1, "hello", True] -- 非法
```

2. 列表操作

拼接

```
[1, 2] ++ [3, 4]  
-- [1, 2, 3, 4]
```

头部插入

```
1 : [2, 3, 4]  
-- [1, 2, 3, 4]
```

注意：

- `:` 是元素接列表

- ++ 是列表接列表

3. 常用列表函数

```
head [1,2,3]      -- 1
tail [1,2,3]      -- [2,3]
length [1,2,3]    -- 3
null []           -- True
reverse [1,2,3]    -- [3,2,1]
take 2 [1,2,3,4]  -- [1,2]
drop 2 [1,2,3,4]  -- [3,4]
sum [1,2,3]        -- 6
product [1,2,3]    -- 6
```

但是：

```
head []
tail []
```

会报错。

这提醒我们：某些函数并不安全，后面我们会用 `Maybe` 来改进这种设计。

4. 范围与无限列表

```
[1..5]
-- [1,2,3,4,5]

['a'..'f']
-- "abcdef"

[2,4..10]
-- [2,4,6,8,10]
```

无限列表：

```
[1..]
```

配合 `take`：

```
take 10 [1..]
```

结果：

```
[1,2,3,4,5,6,7,8,9,10]
```

5. 列表推导式

```
[x * 2 | x <- [1..5]]  
-- [2,4,6,8,10]
```

带条件：

```
[x | x <- [1..10], even x]  
-- [2,4,6,8,10]
```

多个生成器：

```
[(x, y) | x <- [1,2], y <- ['a','b']]  
-- [(1,'a'),(1,'b'),(2,'a'),(2,'b')]
```

6. 元组

```
person :: (String, Int)  
person = ("Alice", 25)
```

访问二元组：

```
fst ("Alice", 25)  
snd ("Alice", 25)
```

结果：

```
"Alice"  
25
```

7. 模式匹配

常量匹配

```
sayNumber :: Int -> String
sayNumber 1 = "one"
sayNumber 2 = "two"
sayNumber _ = "other"
```

列表匹配

```
myHead :: [a] -> a
myHead (x:_) = x
```

更完整：

```
describeList :: [a] -> String
describeList [] = "empty"
describeList [_] = "one element"
describeList (_:_:_) = "multiple elements"
```

8. 递归

阶乘

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

列表求和

```
sumList :: [Int] -> Int
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

列表长度

```
myLength :: [a] -> Int
myLength [] = 0
myLength (_:xs) = 1 + myLength xs
```

9. 守卫

```
grade :: Int -> String
grade score
| score >= 90 = "A"
| score >= 80 = "B"
| score >= 70 = "C"
| score >= 60 = "D"
| otherwise  = "F"
```

10. where 子句

```
bmiTell :: Double -> Double -> String
bmiTell weight height
| bmi <= 18.5 = "Underweight"
| bmi <= 25.0 = "Normal"
| bmi <= 30.0 = "Overweight"
| otherwise  = "Obese"
where
    bmi = weight / (height ^ 2)
```

11. Day 2 练习

实现 `myNull`

```
myNull :: [a] -> Bool
myNull [] = True
myNull _  = False
```

实现 `myProduct`

```
myProduct :: [Int] -> Int
myProduct [] = 1
myProduct (x:xs) = x * myProduct xs
```

实现 `oddNumbers`

```
oddNumbers :: [Int] -> [Int]
oddNumbers xs = [x | x <- xs, odd x]
```

Day 3：高阶函数、柯里化、lambda、组合与 fold

今天开始，真正进入函数式编程的核心。

1. 函数是一等公民

在 Haskell 中，函数和数字、字符串一样，都是值。

这意味着函数可以：

- 作为参数传入其他函数
- 作为返回值返回
- 被组合
- 被部分应用

这就是高阶函数的基础。

2. map

```
map :: (a -> b) -> [a] -> [b]
```

例子：

```
map (+1) [1,2,3,4]
-- [2,3,4,5]
```

```
map length ["hi", "haskell", "abc"]
-- [2,7,3]
```

自己实现：

```
myMap :: (a -> b) -> [a] -> [b]
myMap _ [] = []
myMap f (x:xs) = f x : myMap f xs
```

3. filter

```
filter :: (a -> Bool) -> [a] -> [a]
```

例子：

```
filter even [1..10]  
-- [2,4,6,8,10]
```

实现：

```
myFilter :: (a -> Bool) -> [a] -> [a]  
myFilter _ [] = []  
myFilter p (x:xs)  
| p x      = x : myFilter p xs  
| otherwise = myFilter p xs
```

4. 柯里化

```
add :: Int -> Int -> Int  
add x y = x + y
```

本质上可理解为：

```
add :: Int -> (Int -> Int)
```

所以：

```
add 3
```

返回的是一个函数。

```
add3 :: Int -> Int  
add3 = add 3
```

调用：

```
add3 10
-- 13
```

5. 部分应用

```
multiply :: Int -> Int -> Int
multiply x y = x * y

double :: Int -> Int
double = multiply 2

triple :: Int -> Int
triple = multiply 3
```

部分应用是 Haskell 风格中非常自然的一部分。

6. lambda 匿名函数

```
\x -> x + 1
```

示例：

```
map (\x -> x * x) [1,2,3,4]
-- [1,4,9,16]
```

```
filter (\x -> x > 10) [3,8,12,15]
-- [12,15]
```

7. foldr：抽象递归模式

很多列表递归都可以抽象成 fold。

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

例子：

```
foldr (+) 0 [1,2,3,4]
-- 10
```

展开：

```
1 + (2 + (3 + (4 + 0)))
```

实现：

```
myFoldr :: (a -> b -> b) -> b -> [a] -> b
myFoldr _ acc [] = acc
myFoldr f acc (x:xs) = f x (myFoldr f acc xs)
```

8. 用 foldr 实现常见函数

sum

```
sumWithFold :: [Int] -> Int
sumWithFold = foldr (+) 0
```

length

```
lengthWithFold :: [a] -> Int
lengthWithFold = foldr (\_ acc -> acc + 1) 0
```

and

```
myAnd :: [Bool] -> Bool
myAnd = foldr (&&) True
```

9. 函数组合

```
(.) :: (b -> c) -> (a -> b) -> a -> c
```

例子：

```
f = (*2)
g = (+1)

h = f . g
```

```
h 3
-- 8
```

因为先 $(+1)$ ，再 $(*2)$ 。

10. point-free 风格

普通写法：

```
addOneToAll xs = map (+1) xs
```

point-free 写法：

```
addOneToAll = map (+1)
```

再比如：

```
countEvens xs = length (filter even xs)
```

可以写成：

```
countEvens = length . filter even
```

注意：不要为了炫技而过度 point-free。

11. let 表达式

```
calculate =
  let x = 10
      y = 20
  in x + y
```

12. Day 3 练习

统计大于 10 的元素个数

```
countGreaterThan10 :: [Int] -> Int
countGreaterThan10 = length . filter (>10)
```

平方后取偶数

```
squareEvens :: [Int] -> [Int]
squareEvens = map (^2) . filter even
```

Day 4: 代数数据类型、记录、类型别名、Maybe 与 Either

今天是类型驱动设计的起点。

1. 自定义数据类型

```
data Color = Red | Green | Blue
    deriving (Show)
```

使用：

```
favorite :: Color
favorite = Red
```

2. 带参数构造器

```
data Point = Point Double Double
    deriving (Show)
```

```
distanceFromOrigin :: Point -> Double
distanceFromOrigin (Point x y) = sqrt (x * x + y * y)
```

3. 记录语法

```
data Person = Person
    { name :: String
    , age  :: Int
    } deriving (Show)
```

创建：

```
alice = Person { name = "Alice", age = 25 }
```

4. 类型别名

```
type Name = String
type Age = Int
type PersonInfo = (Name, Age)
```

5. 参数化类型

```
data Box a = Box a
  deriving (Show)
```

6. Maybe

```
data Maybe a = Nothing | Just a
```

安全头部：

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x:_) = Just x
```

使用：

```
describeHead :: Show a => [a] -> String
describeHead xs =
  case safeHead xs of
    Nothing -> "Empty list"
    Just x   -> "Head is " ++ show x
```

7. Either

```
data Either a b = Left a | Right b
```

安全除法：

```
safeDiv :: Double -> Double -> Either String Double
safeDiv _ 0 = Left "Division by zero"
safeDiv x y = Right (x / y)
```

8. 递归数据结构：二叉树

```
data Tree a
  = Empty
  | Node a (Tree a) (Tree a)
  deriving (Show)
```

```
treeSize :: Tree a -> Int
treeSize Empty = 0
treeSize (Node _ left right) = 1 + treeSize left + treeSize right
```

```
treeHeight :: Tree a -> Int
treeHeight Empty = 0
treeHeight (Node _ left right) =
  1 + max (treeHeight left) (treeHeight right)
```

```
inorder :: Tree a -> [a]
inorder Empty = []
inorder (Node x left right) = inorder left ++ [x] ++ inorder right
```

9. data、type、newtype 的区别

这是非常值得初学者尽早建立的概念。

type

只是别名，不创建新类型。

```
type UserId = Int
```

data

创建真正的新类型，可以有多个构造器。

```
data UserId = UserId Int
```

newtype

也创建新类型，但只能有一个构造器、一个字段，运行时开销更低。

```
newtype UserId = UserId Int
```

初学阶段你可以先记住：

- 想要语义别名：`type`
- 想要真正区分类型：`data` / `newtype`

10. Day 4 练习

定义星期类型

```
data Weekday = Mon | Tue | Wed | Thu | Fri | Sat | Sun
    deriving (Show, Eq)
```

判断是否周末

```
isWeekend :: Weekday -> Bool
isWeekend Sat = True
isWeekend Sun = True
isWeekend _   = False
```

安全最大值

```
safeMaximum :: Ord a => [a] -> Maybe a
safeMaximum [] = Nothing
safeMaximum [x] = Just x
safeMaximum (x:xs) =
    case safeMaximum xs of
        Nothing -> Just x
        Just m   -> Just (max x m)
```

Day 5: 类型类、Functor、Applicative、Monad 的直觉与实践

这一部分是很多初学者的“心理门槛”。
我们不从抽象数学定义开始，而是从代码和直觉开始。

1. 类型类是什么？

类型类不是 Java/C++ 里的“类”。
它更像是“一组类型必须实现的行为”。

例如：

```
(==) :: Eq a => a -> a -> Bool
```

意思是：

- 对任意类型 `a`
- 只要它属于 `Eq`
- 就可以用 `==`

常见类型类：

- `Eq`
 - `Ord`
 - `Show`
 - `Read`
 - `Num`
 - `Functor`
 - `Applicative`
 - `Monad`
-

2. deriving 自动生成实例

```
data Color = Red | Green | Blue
deriving (Show, Eq, Ord)
```

这样就可以：

```
show Red
Red == Blue
```

```
Red < Blue
```

3. Functor：对上下文中的值做映射

```
fmap :: (a -> b) -> f a -> f b
```

列表上的 Functor

```
fmap (+1) [1,2,3]  
-- [2,3,4]
```

Maybe 上的 Functor

```
fmap (+1) (Just 3)  
-- Just 4  
  
fmap (+1) Nothing  
-- Nothing
```

直觉：

Functor 让你把一个普通函数应用到“包裹在某种上下文中的值”上。

4. Applicative：上下文中的函数应用

```
pure  :: a -> f a  
(<*>) :: f (a -> b) -> f a -> f b
```

Maybe 例子

```
pure 3 :: Maybe Int  
-- Just 3
```

```
Just (+1) <*> Just 3  
-- Just 4
```

多参数函数

```
add :: Int -> Int -> Int
add x y = x + y
```

```
add <$> Just 3 <*> Just 5
-- Just 8
```

如果某一步失败：

```
add <$> Just 3 <*> Nothing
-- Nothing
```

5. Monad：串联带上下文的计算

核心接口：

```
(>=) :: m a -> (a -> m b) -> m b
```

Monad 的直觉可以理解为：

当前一步计算的结果会影响下一步，并且每一步都在某种上下文中时，Monad 可以把它们优雅串联起来。

6. 用 Maybe 理解 Monad

```
safeReciprocal :: Double -> Maybe Double
safeReciprocal 0 = Nothing
safeReciprocal x = Just (1 / x)

safeRoot :: Double -> Maybe Double
safeRoot x
  | x < 0      = Nothing
  | otherwise = Just (sqrt x)
```

不用 Monad

```
safeOp :: Double -> Maybe Double
safeOp x =
```

```
case safeReciprocal x of
  Nothing -> Nothing
  Just r  ->
    case safeRoot r of
      Nothing -> Nothing
      Just y  -> Just y
```

用 bind

```
safeOp2 :: Double -> Maybe Double
safeOp2 x = safeReciprocal x >=> safeRoot
```

7. do 语法

```
safeChainDo :: Double -> Maybe Double
safeChainDo x = do
  r <- safeReciprocal x
  y <- safeRoot r
  return y
```

注意：`return` 在 Haskell 中不是“退出函数”，而是“把值放回 Monad 上下文”。

8. Either Monad

`Either` 常用于错误处理。

```
safeReciprocalE :: Double -> Either String Double
safeReciprocalE 0 = Left "Cannot take reciprocal of zero"
safeReciprocalE x = Right (1 / x)

safeRootE :: Double -> Either String Double
safeRootE x
  | x < 0      = Left "Cannot sqrt negative number"
  | otherwise = Right (sqrt x)
```

串联：

```
safeOpE :: Double -> Either String Double
safeOpE x = do
  r <- safeReciprocalE x
  y <- safeRootE r
  return y
```

9. 列表 Monad

```
pairs :: [(Int, Int)]
pairs = do
  x <- [1,2]
  y <- [10,20]
  return (x, y)
```

结果：

```
[(1,10),(1,20),(2,10),(2,20)]
```

10. Monad 的正确入门姿势

不要一上来硬背抽象定义。

先建立这几个对应关系：

- **Maybe**：可能失败
- **Either**：可能失败且带错误信息
- **[]**：多结果计算
- **IO**：副作用计算

当你能在这几个具体实例里熟练使用 **do**、**>>=** 和 **return**，你对 Monad 的理解就已经有了坚实基础。

Day 6: IO、do 语法、文件读写与纯/不纯边界

今天我们来解决一个关键问题：

Haskell 是纯函数式语言，那它怎么和现实世界交互？

答案是：通过把副作用显式放进类型系统。

1. IO 的直觉

看一个最简单的例子：

```
putStrLn "Hello"
```

类型：

```
putStrLn :: String -> IO ()
```

这说明它不是单纯返回 `()`，而是返回一个 `IO ()`。
可以理解为：它描述了一个带副作用的动作。

2. main 函数

```
main :: IO ()  
main = putStrLn "Hello, Haskell!"
```

程序入口就是一个 `IO ()` 动作。

3. 获取用户输入

```
main :: IO ()  
main = do  
    putStrLn "What is your name?"  
    name <- getLine  
    putStrLn ("Hello, " ++ name)
```

看类型：

```
getLine :: IO String
```

这意味着：

- `getLine` 不是字符串
- 它是一个执行后会产生字符串的 IO 动作

4. do 块中的两类语句

只执行动作，不取结果

```
putStrLn "Hello"
```

执行动作并绑定结果

```
name <- getLine
```

如果你写：

```
name = getLine
```

那么 `name` 的类型是 `IO String`，不是 `String`。

5. 一个简单计算器

```
main :: IO ()
main = do
  putStrLn "Enter first number:"
  aStr <- getLine
  putStrLn "Enter second number:"
  bStr <- getLine

  let a = read aStr :: Int
      b = read bStr :: Int

  putStrLn ("Sum = " ++ show (a + b))
```

6. `read` 的风险与安全解析意识

`read` 很方便，但它不是安全函数：

```
read "123" :: Int
```

没问题。

但：

```
read "abc" :: Int
```

会抛异常。

这说明在真实项目中，我们需要更安全的解析方式。

初学阶段你可以先知道这一点，后面逐步学习 `reads`、`Text.Read.readMaybe` 等更安全方案。

例如使用 `readMaybe`：

```
import Text.Read (readMaybe)

parseInt :: String -> Maybe Int
parseInt = readMaybe
```

使用：

```
safeAddInput :: String -> String -> Maybe Int
safeAddInput aStr bStr = do
  a <- readMaybe aStr
  b <- readMaybe bStr
  return (a + b)
```

7. 文件读写

读取文件

```
main :: IO ()
main = do
  content <- readFile "test.txt"
  putStrLn content
```

写入文件

```
main :: IO ()
main = do
  writeFile "output.txt" "Hello from Haskell!"
  putStrLn "Done."
```

追加写入

```
main :: IO ()
main = do
  appendFile "log.txt" "new line\n"
  putStrLn "Appended."
```

8. 纯函数与 IO 分离

这是 Haskell 工程实践中最重要的原则之一。

不推荐

```
main :: IO ()
main = do
  line <- getLine
  let nums = map read (words line) :: [Int]
  print (sum nums)
```

推荐

```
sumNumbers :: String -> Int
sumNumbers input = sum (map read (words input))

main :: IO ()
main = do
  line <- getLine
  print (sumNumbers line)
```

这样：

- `sumNumbers` 是纯函数，容易测试
- `main` 只负责交互
- 程序结构更清晰

9. 一个简单的 TODO 查看器

```
formatTodos :: String -> String
formatTodos content =
  unlines $
    zipWith (\i line -> show i ++ ". " ++ line) [1..] (lines content)

main :: IO ()
main = do
  content <- readFile "todo.txt"
  putStrLn "TODO List:"
  putStrLn (formatTodos content)
```

10. IO 与 Monad 的关系

IO 本身也是 Monad。

这意味着 `do` 语法同样适用于 IO。

你可以把它和 `Maybe` 的 `do` 对比理解：

- `Maybe` 的 `do`：可能在某一步失败
 - `IO` 的 `do`：每一步都是副作用动作，按顺序执行
-

Day 7: 模块化、Cabal/Stack、项目结构与实战案例

学到这里，你已经具备写小程序的能力了。
接下来我们从“单文件练习”过渡到“项目组织”。

1. 模块系统

定义模块：

```
module MathUtils where

add :: Int -> Int -> Int
add x y = x + y

square :: Int -> Int
square x = x * x
```

另一个文件：

```
module Main where

import MathUtils

main :: IO ()
main = do
    print (add 3 4)
    print (square 5)
```

2. 选择性导入与隐藏

```
import MathUtils (add)
```

```
import Prelude hiding (sum)
```

3. Cabal 项目

创建项目：

```
cabal init
```

构建：

```
cabal build
```

运行：

```
cabal run
```

进入 REPL：

```
cabal repl
```

典型结构：

```
my-project/  
├─ app/  
│   └─ Main.hs  
├─ src/  
│   └─ Lib.hs  
├─ test/  
└─ my-project.cabal
```

4. Stack 项目

创建：

```
stack new my-project  
cd my-project  
stack build  
stack run  
stack ghci
```

5. 实战案例：成绩分析器

`src/GradeAnalyzer.hs`

```

module GradeAnalyzer where

parseScores :: String -> [Int]
parseScores input = map read (words input)

totalScore :: [Int] -> Int
totalScore = sum

averageScore :: [Int] -> Double
averageScore [] = 0
averageScore xs = fromIntegral (sum xs) / fromIntegral (length xs)

maxScore :: [Int] -> Maybe Int
maxScore [] = Nothing
maxScore xs = Just (maximum xs)

countFailed :: [Int] -> Int
countFailed = length . filter (< 60)

gradeLabel :: Int -> String
gradeLabel score
  | score >= 90 = "A"
  | score >= 80 = "B"
  | score >= 70 = "C"
  | score >= 60 = "D"
  | otherwise  = "F"

gradeDistribution :: [Int] -> [(String, Int)]
gradeDistribution xs =
  [ ("A", count "A")
  , ("B", count "B")
  , ("C", count "C")
  , ("D", count "D")
  , ("F", count "F")
  ]
  where
    labels = map gradeLabel xs
    count g = length (filter (== g) labels)

formatReport :: [Int] -> String
formatReport xs =
  unlines
    [ "Total: " ++ show (totalScore xs)
    , "Average: " ++ show (averageScore xs)
    , "Max: " ++ show (maxScore xs)
    , "Failed: " ++ show (countFailed xs)
    , "Distribution: " ++ show (gradeDistribution xs)
    ]

```

```
module Main where

import GradeAnalyzer

main :: IO ()
main = do
    putStrLn "Enter scores separated by spaces:"
    line <- getLine
    let scores = parseScores line
    putStrLn "Analysis Report:"
    putStrLn (formatReport scores)
```

6. 代码分析

这个例子体现了 Haskell 的几个核心工程实践：

1) 纯逻辑与 IO 分离

所有分析逻辑都在纯函数中。

2) 类型驱动设计

`maxScore :: [Int] -> Maybe Int` 明确表达了空列表情况。

3) 组合式编程

`countFailed = length . filter (< 60)`

四、惰性求值深讲：Haskell 为什么能处理无限列表？

惰性求值是 Haskell 极其重要的特征之一。

如果不理解它，你会对很多代码行为感到困惑。

1. 什么是惰性求值？

惰性求值的核心思想：

表达式不会在定义时立即计算，而是在真正需要结果时才计算。

例如：

```
x = 1 + 2
```

这里 `x` 不是立刻被算成 `3`，而是可以理解为“一个待求值的表达式”。

2. 为什么无限列表可行？

```
[1..]
```

这是一个无限列表。

在严格求值语言中，这样的表达式通常会立刻卡死。

但在 Haskell 中：

```
take 5 [1..]
```

只会按需取前 5 个元素。

3. 惰性带来的表达力

无限数据结构

```
naturals :: [Integer]
naturals = [1..]

evens :: [Integer]
evens = map (*2) naturals
```

```
take 10 evens
-- [2,4,6,8,10,12,14,16,18,20]
```

按需组合计算

```
take 5 (filter even (map (^2) [1..]))
```

4. 短路行为

```
False && error "boom"
```

结果是：

False

因为右边根本不要求值。

5. 惰性求值的代价：空间泄漏风险

惰性求值会保留尚未计算的表达式，这些延迟计算结构常被称为 thunk。
如果 thunk 大量堆积，可能导致内存问题。

例如某些情况下：

```
foldl (+) 0 [1..1000000]
```

可能不如严格版本高效。

更常见的做法是：

```
import Data.List (foldl')

sumStrict :: [Int] -> Int
sumStrict = foldl' (+) 0
```

6. 初学者对惰性的正确理解方式

先记住这几点就够了：

- Haskell 默认按需计算
 - 这让无限列表成为可能
 - 这让很多组合写法很自然
 - 但也意味着你要逐渐关注求值时机和空间使用
-

五、Haskell 初学者常见误区与踩坑指南

1. 把 `IO String` 当成 `String`

错误：

```
name = getLine
```

正确：

```
main = do
  name <- getLine
  putStrLn name
```

2. 以为 `return` 是“函数返回”

不是。

它是“把值放进 Monad 上下文”。

3. 过早纠结 Monad 定义

先会用 `Maybe`、`Either`、`IO` 即可。

4. 不写类型签名

短期省事，长期痛苦。

类型签名是文档、约束和设计工具。

5. 滥用 `point-free`

过度 `point-free` 会让代码难读。

清晰性优先。

6. 只会调用库函数，不会手写递归

这会导致你对本质理解不足。

入门阶段一定要手写：

- `myMap`
- `myFilter`

- `myFoldr`
 - `myLength`
 - `sumList`
-

7. 直接使用不安全函数

如：

- `head`
- `tail`
- `read`

真实项目中要有安全意识。

8. 把所有逻辑写进 `main`

这会损失 Haskell 最大的工程优势之一：纯函数可测试。

9. 害怕模式匹配

模式匹配是 Haskell 代码最自然的结构方式之一，不要回避。

10. 觉得 Haskell 只适合“学术”

这是一种误解。

虽然 Haskell 的生态规模不如 Java/Python/JS，但它在编译器、金融、DSL、区块链、后端服务等领域都有实际应用。

六、如何高效学习 Haskell：方法、建议与路线图

1. 多用 GHCi

多做这些操作：

```
:t map
:t foldr
:t Just
:t Nothing
:t putStrLn
```

类型是理解 Haskell 的第一入口。

2. 先会手写，再会抽象

比如先手写递归版 `sumList`，再理解 `foldr`。
这样你会真正知道抽象来自哪里。

3. 通过小项目巩固

推荐练习：

- 交互式计算器
 - TODO CLI
 - 文本词频统计器
 - 成绩分析器
 - 文件行号工具
 - 二叉树操作器
 - 表达式求值器
-

4. 学会看类型推导程序结构

例如：

```
safeHead :: [a] -> Maybe a
```

这就已经告诉你：

- 输入是列表
- 输出可能有值也可能没有
- 空列表必须处理

5. 接受“理解是分层到来的”

很多概念你第一次看只是“认识”，第二次看是“理解”，第三次看才是“会用”。

尤其是：

- 类型类
- Applicative
- Monad
- 惰性求值

6. 推荐后续进阶路线

第一阶段：夯实基础

- 模式匹配
- 递归
- 高阶函数
- ADT
- Maybe / Either
- IO

第二阶段：工程化

- Cabal / Stack
- 测试
- Text / ByteString
- Aeson
- Containers

第三阶段：高级抽象

- Monad Transformer
 - Reader / State / Writer
 - Traversable / Foldable
 - GADT
 - Type Families
 - Lens
-

七、结语：Haskell 真正改变你的是什么？

Haskell 的价值，不只是让你学会一门“小众但优雅”的语言。
它真正改变的是你看待程序的方式。

你会越来越重视：

- 纯函数
- 显式副作用
- 类型驱动设计
- 不可变数据
- 可组合的抽象
- 错误显式化
- 程序可推理性

即便你最终主要使用的是其他语言，Haskell 带给你的这些能力，依然会深刻影响你的工程判断和代码质量。

从这个意义上说，Haskell 不是一门“只为写 Haskell 而学”的语言。
它更像是一门帮你理解“程序本质”的语言。

附录 A：完整代码清单

基础函数

```
add :: Int -> Int -> Int
add a b = a + b

square :: Int -> Int
square x = x * x

isEven :: Int -> Bool
isEven x = x `mod` 2 == 0
```

递归

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)

sumList :: [Int] -> Int
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

高阶函数

```
myMap :: (a -> b) -> [a] -> [b]
myMap _ [] = []
myMap f (x:xs) = f x : myMap f xs

myFilter :: (a -> Bool) -> [a] -> [a]
myFilter _ [] = []
myFilter p (x:xs)
  | p x      = x : myFilter p xs
  | otherwise = myFilter p xs
```

foldr

```
myFoldr :: (a -> b -> b) -> b -> [a] -> b
myFoldr _ acc [] = acc
myFoldr f acc (x:xs) = f x (myFoldr f acc xs)
```

Maybe

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x:_) = Just x
```

Either

```
safeDiv :: Double -> Double -> Either String Double
safeDiv _ 0 = Left "Division by zero"
safeDiv x y = Right (x / y)
```

Tree

```
data Tree a
  = Empty
  | Node a (Tree a) (Tree a)
  deriving (Show)

treeSize :: Tree a -> Int
treeSize Empty = 0
treeSize (Node _ left right) = 1 + treeSize left + treeSize right
```

IO

```
main :: IO ()
main = do
  putStrLn "What is your name?"
  name <- getLine
  putStrLn ("Hello, " ++ name)
```

附录 B：练习题与参考答案

练习 1：实现 `myReverse`

```
myReverse :: [a] -> [a]
myReverse [] = []
myReverse (x:xs) = myReverse xs ++ [x]
```

练习 2：实现安全尾部函数

```
safeTail :: [a] -> Maybe [a]
safeTail [] = Nothing
safeTail (_:xs) = Just xs
```

练习 3：统计字符串长度列表

```
stringLengths :: [String] -> [Int]
stringLengths = map length
```

练习 4：用 `Maybe` 串联两个可能失败的解析

```
import Text.Read (readMaybe)

addTwoInputs :: String -> String -> Maybe Int
addTwoInputs aStr bStr = do
  a <- readMaybe aStr
  b <- readMaybe bStr
  return (a + b)
```

练习 5：实现树的前序遍历

```
preorder :: Tree a -> [a]
preorder Empty = []
preorder (Node x left right) = [x] ++ preorder left ++ preorder right
```